```
})
```

There's a lot going on here, some of which is behind the scenes. Let's start with a straightforward translation of what Cypress is doing.

The `beforeEach` function is called before each spec runs. In this case, the function is making a web request to our special `cypress-rails` reset route. As we saw earlier, this resets the database back to its original state.

To test favorite behavior, we need a logged-in user, so we start the test by using `cy.visit("users/sign_in")` to visit our login page. Then we use `cy.get` to retrieve our email form field and password field, and `type` to simulate typing in the email and password. Finally, we find the login button and use `click` to simulate a click. This logs our sample user in, but this process is a bit clunky to do all the time, and we'll explore alternatives later on.

Once we are logged in, we call `cy.visit("/")`, which also, as we saw before, makes a web request to our Rails server and takes us to the schedule page.

Our next two lines identify parts of the page that we will be looking at in this test. First, we call `cy.get("#favorite-concerts-list").as("favorites")`, then we call `cy.get(".concert").first().as("concert")`. Let's unpack this.

We start with `get`, which is one of several dozen commands the Cypress object receives. (The full list can be found online.[81]) The `get` command takes in a CSS selector, which in this case is either `.#favorite-concerts-list`, meaning "the element with the DOM ID `favorite-concerts-list`," or `.concert`, meaning "elements with the DOM class `concert`." The `get` command behaves like the jQuery `$` operator, if that reference is useful for you.

We can then chain more operations on the set of DOM elements that match the selector passed to `get`. I'm deliberately not saying that `get` returns a set of DOM elements, as technically it doesn't because Cypress works asynchronously. More on that in a moment.

If there are no elements matching the selector, Cypress waits for a specified